Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота №4

з дисципліни «Організація баз даних»
на тему «Аналітичні SQL-запити (OLAP)»

ВИКОНАВ:
студент ІІ курсу ФІОТ
групи ІО-44
Шмігель Макар Олексійович

ПЕРЕВІРИВ:
Русінов В. В.

Київ – 2026

Лабораторна робота №4

Тема: Аналітичні SQL-запити (OLAP) у реляційній базі даних PostgreSQL.

Мета: Опанувати принципи онлайн-аналітичної обробки даних (OLAP). Набути практичних навичок написання складних аналітичних SQL-запитів з використанням агрегатних функцій (COUNT, SUM, AVG, MIN, MAX), групування даних (GROUP BY, HAVING) та об'єднання кількох таблиць (INNER JOIN, LEFT JOIN) для отримання зведеної статистики та формування звітів.

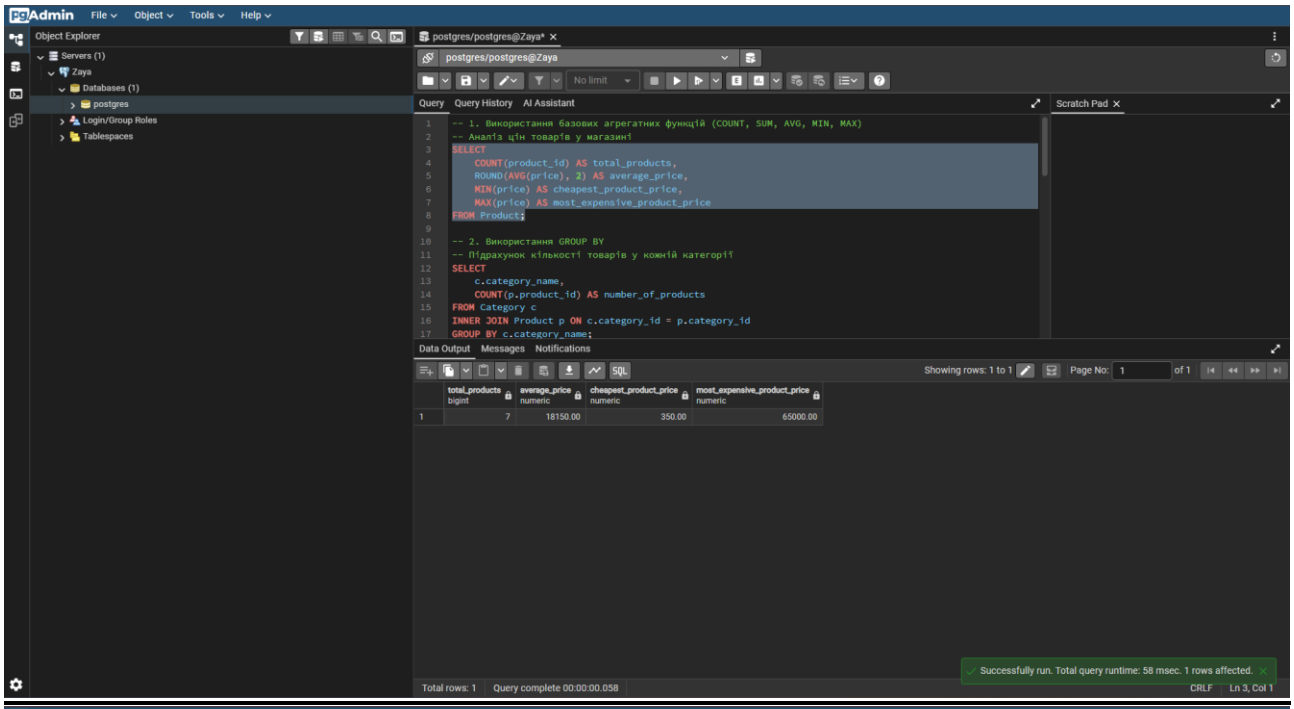
Код:

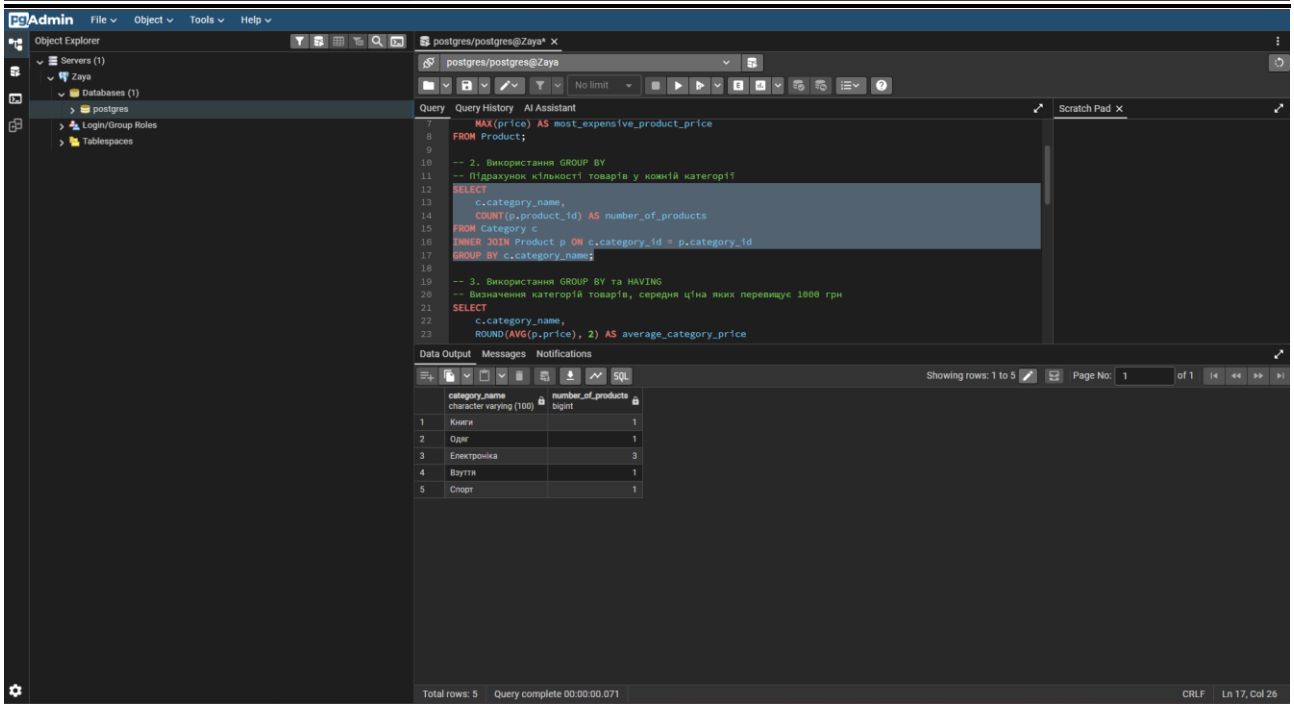
```
-- 1. Використання базових агрегатних функцій (COUNT, SUM, AVG, MIN, MAX)

-- Аналіз цін товарів у магазині

SELECT

    COUNT(product_id) AS total_products,

    ROUND(AVG(price), 2) AS average_price,

    MIN(price) AS cheapest_product_price,

    MAX(price) AS most_expensive_product_price

FROM Product;


-- 2. Використання GROUP BY

-- Підрахунок кількості товарів у кожній категорії

SELECT

    c.category_name,

    COUNT(p.product_id) AS number_of_products

FROM Category c

INNER JOIN Product p ON c.category_id = p.category_id

GROUP BY c.category_name;
```

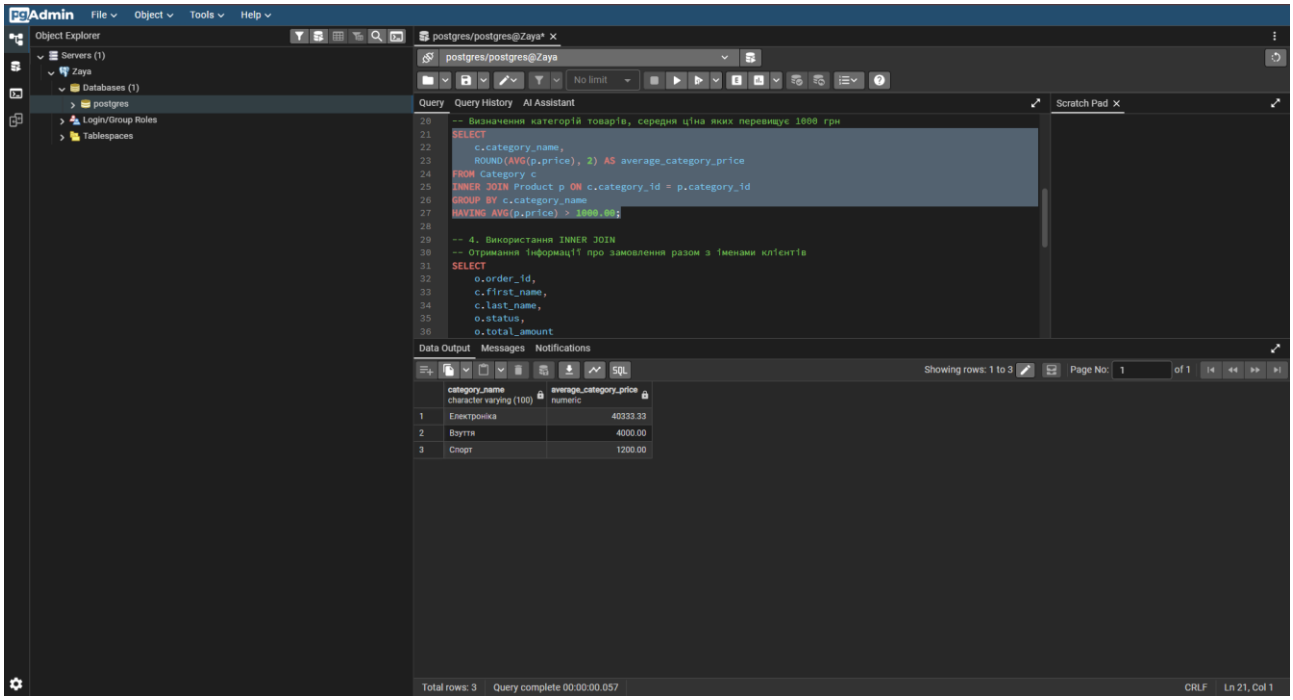
```
-- 3. Використання GROUP BY та HAVING

-- Визначення категорій товарів, середня ціна яких перевищує 1000 грн

SELECT

    c.category_name,

    ROUND(AVG(p.price), 2) AS average_category_price

FROM Category c

INNER JOIN Product p ON c.category_id = p.category_id

GROUP BY c.category_name

HAVING AVG(p.price) > 1000.00;


-- 4. Використання INNER JOIN

-- Отримання інформації про замовлення разом з іменами клієнтів

SELECT

    o.order_id,

    c.first_name,

    c.last_name,

    o.status,

    o.total_amount

FROM Orders o

INNER JOIN Customer c ON o.customer_id = c.customer_id;


-- 5. Використання LEFT JOIN

-- Виведення всіх клієнтів і їхніх замовлень (якщо замовлень немає, буде
NULL)

SELECT

    c.first_name,
```

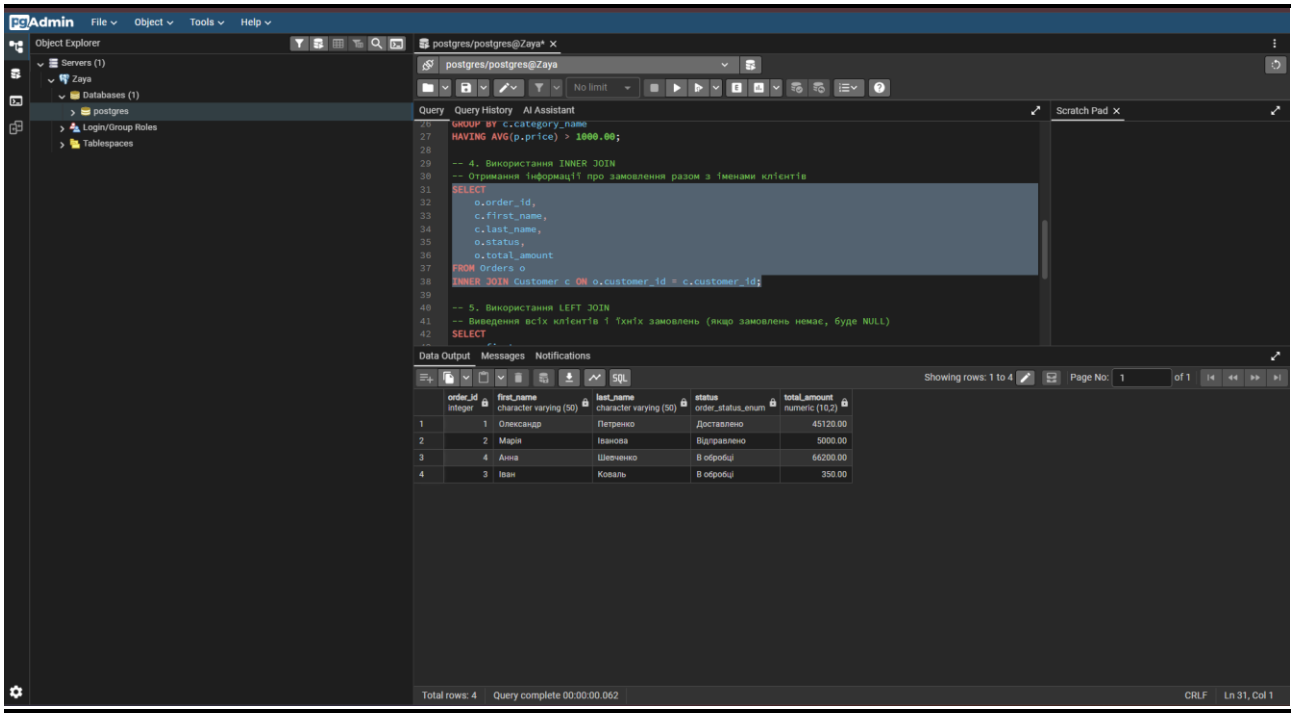
```
    c.last_name,

    o.order_id,

    o.total_amount

FROM Customer c

LEFT JOIN Orders o ON c.customer_id = o.customer_id;


-- 6. Складний об'єднаний запит (JOIN + GROUP BY + SUM + ORDER BY)

-- Рейтинг клієнтів за загальною сумою витрачених коштів (від найбільшого до найменшого)

SELECT

    c.first_name,

    c.last_name,

    SUM(o.total_amount) AS total_spent_money

FROM Customer c

INNER JOIN Orders o ON c.customer_id = o.customer_id

GROUP BY c.first_name, c.last_name

ORDER BY total_spent_money DESC;
```

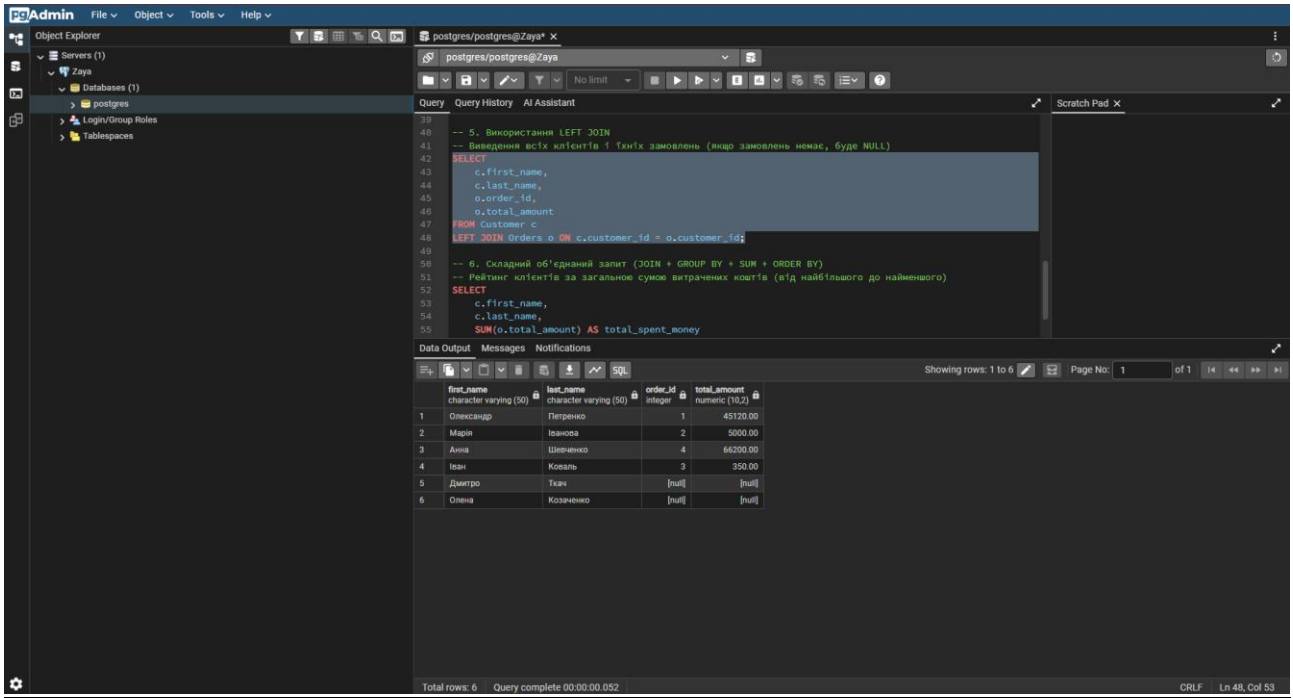
Скріншоти:

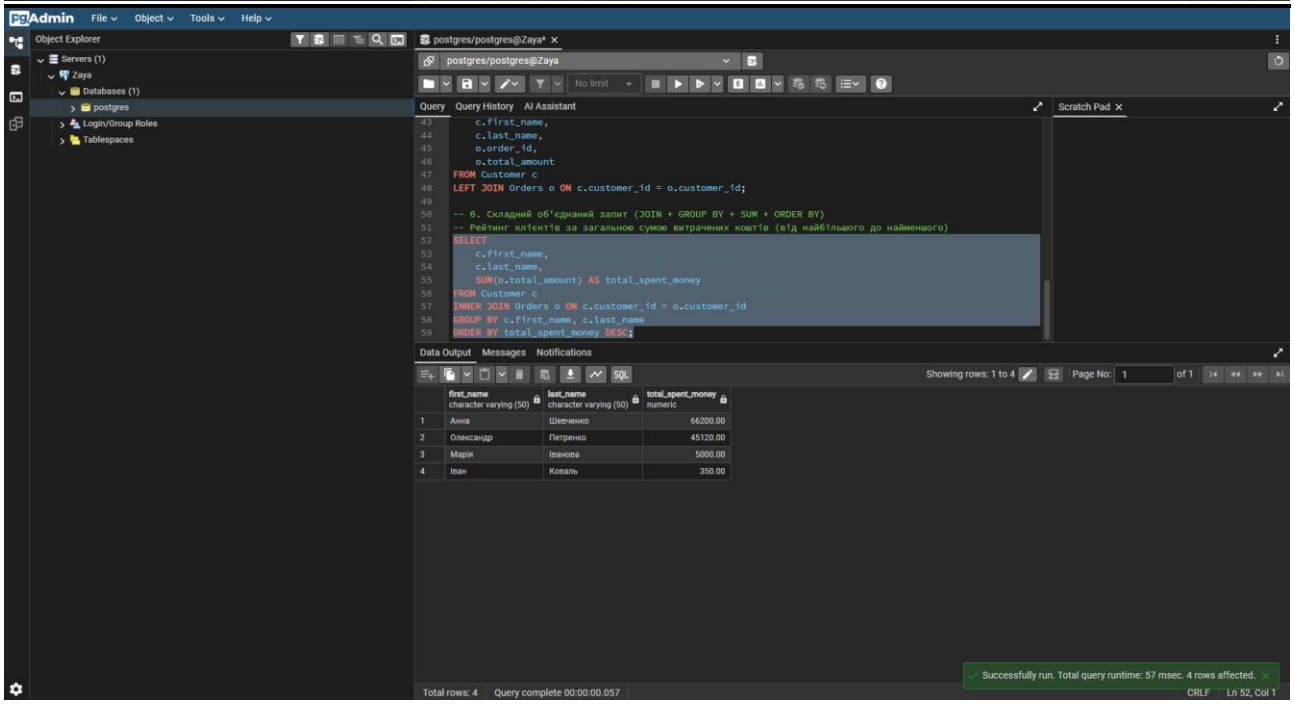

PgAdmin File Object Tools Help
Object Explorer
Servers (1)
Zaya
Databases (1)
postgres
Login/Group Roles
Tablespaces
postgres/postgres@Zaya*
postgres/postgres@Zaya
No limit
Query Query History AI Assistant
Scratch Pad
-- 1. Використання базових агрегатних функцій (COUNT, SUM, AVG, MIN, MAX)
-- Аналіз цін товарів у магазині
SELECT
COUNT(product_id) AS total_products,
ROUND(AVG(price), 2) AS average_price,
MIN(price) AS cheapest_product_price,
MAX(price) AS most_expensive_product_price
FROM Product;
-- 2. Використання GROUP BY
-- Підрахунок кількості товарів у кожній категорії
SELECT
c.category_name,
COUNT(p.product_id) AS number_of_products
FROM Category c
INNER JOIN Product p ON c.category_id = p.category_id
GROUP BY c.category_name;
Data Output Messages Notifications
Showing rows: 1 to 1
Page No: 1 of 1
total_products bigint
average_price numeric
cheapest_product_price numeric
most_expensive_product_price numeric
1 7 18150.00 350.00 65000.00
Successfully run. Total query runtime: 58 msec. 1 rows affected.
Total rows: 1 Query complete 00:00:00.058
CRLF Ln 3, Col 1


PgAdmin File Object Tools Help
Object Explorer
Servers (1)
Zaya
Databases (1)
postgres
Login/Group Roles
Tablespaces
postgres/postgres@Zaya*
postgres/postgres@Zaya
No limit
Query Query History AI Assistant
Scratch Pad
MAX(price) AS most_expensive_product_price
FROM Product;
-- 2. Використання GROUP BY
-- Підрахунок кількості товарів у кожній категорії
SELECT
c.category_name,
COUNT(p.product_id) AS number_of_products
FROM Category c
INNER JOIN Product p ON c.category_id = p.category_id
GROUP BY c.category_name;
-- 3. Використання GROUP BY та HAVING
-- Визначення категорій товарів, середня ціна яких перевищує 1000 грн
SELECT
c.category_name,
ROUND(AVG(p.price), 2) AS average_category_price
Data Output Messages Notifications
Showing rows: 1 to 5
Page No: 1 of 1
category_name character varying (100)
number_of_products bigint
1 Книги 1
2 Одяг 1
3 Електроніка 3
4 Взуття 1
5 Спорт 1
Total rows: 5 Query complete 00:00:00.071
CRLF Ln 17, Col 26


pgAdmin
File
Object
Tools
Help
Object Explorer
Servers (1)
Zaya
Databases (1)
postgres
Login/Group Roles
Tablespaces
postgres/postgres@Zaya*
postgres/postgres@Zaya
No limit
Query
Query History
AI Assistant
Scratch Pad
-- Визначення категорій товарів, середня ціна яких перевищує 1000 грн
SELECT
c.category_name,
ROUND(AVG(p.price), 2) AS average_category_price
FROM Category c
INNER JOIN Product p ON c.category_id = p.category_id
GROUP BY c.category_name
HAVING AVG(p.price) > 1000.00;
-- 4. Використання INNER JOIN
-- Отримання інформації про замовлення разом з іменами клієнтів
SELECT
o.order_id,
c.first_name,
c.last_name,
o.status,
o.total_amount
Data Output
Messages
Notifications
Showing rows: 1 to 3
Page No: 1
of 1
category_name character varying (100)
average_category_price numeric
1 Електроніка 40333.33
2 Взуття 4000.00
3 Спорт 1200.00
Total rows: 3
Query complete 00:00:00.057
CRLF
Ln 21, Col 1


pgAdmin
File
Object
Tools
Help
Object Explorer
Servers (1)
Zaya
Databases (1)
postgres
Login/Group Roles
Tablespaces
postgres/postgres@Zaya*
postgres/postgres@Zaya
No limit
Query
Query History
AI Assistant
Scratch Pad
GROUP BY c.category_name
HAVING AVG(p.price) > 1000.00;
-- 4. Використання INNER JOIN
-- Отримання інформації про замовлення разом з іменами клієнтів
SELECT
o.order_id,
c.first_name,
c.last_name,
o.status,
o.total_amount
FROM Orders o
INNER JOIN Customer c ON o.customer_id = c.customer_id;
-- 5. Використання LEFT JOIN
-- Виведення всіх клієнтів і їхніх замовлень (якщо замовлень немає, буде NULL)
SELECT
Data Output
Messages
Notifications
Showing rows: 1 to 4
Page No: 1
of 1
order_id integer
first_name character varying (50)
last_name character varying (50)
status order_status_enum
total_amount numeric (10,2)
1 1 Олександр Петренко Доставлено 45120.00
2 2 Марія Іванова Відправлено 5000.00
3 4 Анна Шевченко В обробці 66200.00
4 3 Іван Коваль В обробці 350.00
Total rows: 4
Query complete 00:00:00.062
CRLF
Ln 31, Col 1





Запити:

Запит 1: Базова агрегація (COUNT, AVG, MIN, MAX)

Що робить: Обчислює загальну кількість унікальних товарів у магазині, їхню середню, мінімальну та максимальну ціну.

Чому: Дозволяє швидко оцінити загальний ціновий діапазон та асортимент магазину.

Запит 2: Групування даних (GROUP BY)

Що робить: З'єднує таблиці `Category` та `Product` і підраховує, скільки товарів належить до кожної категорії.

Чому: Допомагає менеджменту зрозуміти, які категорії товарів найширше представлені на складі.

Запит 3: Фільтрація згрупованих даних (HAVING)

Що робить: Рахує середню ціну товарів у кожній категорії, але виводить лише ті категорії, де середня ціна перевищує 1000 грн.

Чому: Дозволяє виокремити преміальні категорії (наприклад, "Електроніка"), відфільтрувавши дешеві категорії.

Запит 4: Внутрішнє з'єднання (INNER JOIN)

Що робить: Об'єднує таблиці `Orders` та `Customer`, щоб замість незрозумілого `customer_id` показати реальне ім'я та прізвище замовника.

Чому: Необхідно для формування читабельного списку замовлень для служби підтримки чи кур'єрів.

Запит 5: Ліве з'єднання (LEFT JOIN)

Що робить: Виводить абсолютно всіх клієнтів з таблиці `Customer` та приєднує до них їхні замовлення. Якщо клієнт щойно зареєструвався і ще нічого не купив, навпроти його імені стоятиме значення `NULL`.

Чому: Цей аналітичний запит ідеально підходить для пошуку клієнтів, які давно не робили покупок, щоб надіслати їм промокод.

Запит 6: Комплексний запит (JOIN + GROUP BY + ORDER BY)

Що робить: Рахує загальну суму покупок (`SUM`) для кожного клієнта і сортує список за спаданням (`ORDER BY DESC`).

Чому: Використовується для визначення "VIP-клієнтів", які приносять магазину найбільше прибутку.

Висновок: У ході виконання лабораторної роботи було успішно застосовано методи онлайн-аналітичної обробки даних (OLAP) до спроектованої реляційної

бази даних (на прикладі Інтернет-магазину). На практиці було закріплено навички написання складних SQL-запитів: виконано обчислення зведеної статистики за допомогою агрегатних функцій, реалізовано групування записів та фільтрацію згрупованих результатів. Крім того, успішно застосовано механізми об'єднання даних з кількох пов'язаних таблиць за допомогою операторів INNER JOIN та LEFT JOIN. Написані запити дозволяють ефективно аналізувати дані, виявляти тенденції та формувати змістовні звіти для бізнес-аналітики.